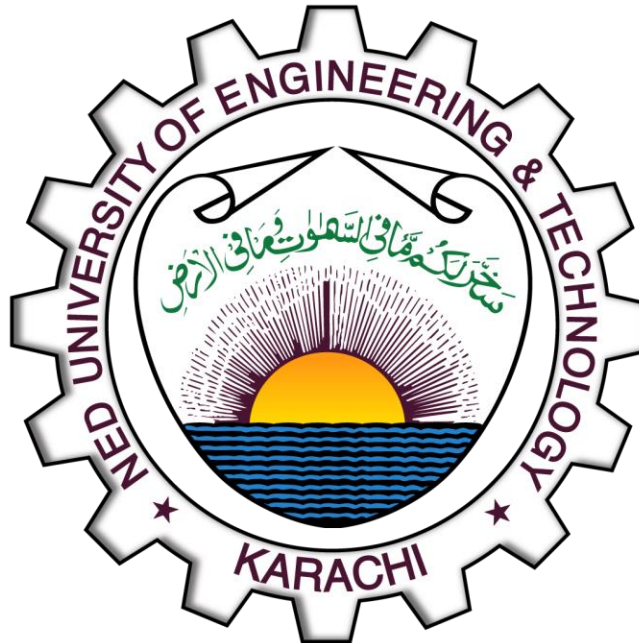


OBJECT ORIENTED PROGRAMMING

[CT-260]



NAME: TAHA AHMED MALLICK

ROLL NO: CT-25183

DEPARTMENT: BCIT **BATCH:** 2025

YEAR & SECTION: FSCS-D

LAB :07

QUESTION#1:

CODE:

```
#include <iostream>
using namespace std;

class Matrix
{
private:
    int **arr, rows, cols;

public:
    Matrix() : rows(0), cols(0), arr(nullptr) {}
    Matrix(int r, int c) : rows(r), cols(c)
    {
        arr = new int *[rows];
        for (int i = 0; i < rows; i++)
            arr[i] = new int[cols]();
    }

    Matrix(const Matrix &other) : rows(other.rows), cols(other.cols)
    { // deep copy
        arr = new int *[rows];
        for (int i = 0; i < rows; i++)
        {
            arr[i] = new int[cols];
            for (int j = 0; j < cols; j++)
                arr[i][j] = other.arr[i][j];
        }
    }

    void operator=(const Matrix &other)
    { // deep copy assignment
        if (this == &other)
            return; // self-assignment check

        for (int i = 0; i < rows; i++)
            delete[] arr[i];
        delete[] arr;

        rows = other.rows;
        cols = other.cols;
        arr = new int *[rows];
        for (int i = 0; i < rows; i++)
        {
            arr[i] = new int[cols];
```

```

        for (int j = 0; j < cols; j++)
            arr[i][j] = other.arr[i][j];
    }
}

Matrix operator+(const Matrix &other)
{
    if (rows != other.rows || cols != other.cols)
    {
        cout << "Matrices must have the same dimensions for addition." << endl;
        return Matrix(0, 0);
    }

    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            result.arr[i][j] = arr[i][j] + other.arr[i][j];
    return result;
}

Matrix operator-(const Matrix &other)
{
    if (rows != other.rows || cols != other.cols)
    {
        cout << "Matrices must have the same dimensions for subtraction." <<
endl;
        return Matrix(0, 0);
    }

    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            result.arr[i][j] = arr[i][j] - other.arr[i][j];
    return result;
}

Matrix operator*(const Matrix &other)
{
    if (cols != other.rows)
    {
        cout << "Number of columns of first matrix must equal number of rows of
second matrix for multiplication." << endl;
        return Matrix(0, 0);
    }

    Matrix result(rows, other.cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < other.cols; j++)

```

```

        for (int k = 0; k < cols; k++)
            result.arr[i][j] += arr[i][k] * other.arr[k][j];
    return result;
}

int *operator[](int index)
{
    return arr[index];
}

~Matrix()
{
    for (int i = 0; i < rows; i++)
    {
        delete[] arr[i];
    }
    delete[] arr;
}

void input()
{
    cout << "Enter elements for a " << rows << "x" << cols << " matrix:" << endl;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> arr[i][j];
}

void print()
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
}

};

int main()
{
    Matrix A(2, 2), B(2, 2);
    A.input();
    B.input();

    Matrix C = A + B;
    cout << "A + B:" << endl;
    C.print();
}

```

```

C = A - B;
cout << "A - B:" << endl;
C.print();

C = A * B;
cout << "A * B:" << endl;
C.print();

cout << "C[0] addr = " << C[0] << endl;
cout << "C[0][0]: " << C[0][0] << endl;
return 0;
}

```

OUTPUT:

```

Enter elements for a 2x2 matrix:
1 2
3 4
Enter elements for a 2x2 matrix:
5 6
7 8
A + B:
6 8
10 12
A - B:
-4 -4
-4 -4
A * B:
19 22
43 50
C[0] addr = 0x5e10ebda3c10
C[0][0]: 19

```

QUESTION#2:

CODE:

```
#include <iostream>
using namespace std;

class Matrix
{
private:
    int **arr, rows, cols;

public:
    Matrix() : rows(0), cols(0), arr(nullptr) {}
    Matrix(int r, int c) : rows(r), cols(c)
    {
        arr = new int *[rows];
        for (int i = 0; i < rows; i++)
            arr[i] = new int[cols]();
    }

    Matrix(const Matrix &other) : rows(other.rows), cols(other.cols)
    { // deep copy
        arr = new int *[rows];
        for (int i = 0; i < rows; i++)
        {
            arr[i] = new int[cols];
            for (int j = 0; j < cols; j++)
                arr[i][j] = other.arr[i][j];
        }
    }

    void operator=(const Matrix &other)
    { // deep copy assignment
        if (this == &other)
            return; // self-assignment check

        for (int i = 0; i < rows; i++)
            delete[] arr[i];
        delete[] arr;

        rows = other.rows;
        cols = other.cols;
        arr = new int *[rows];
        for (int i = 0; i < rows; i++)
        {
            arr[i] = new int[cols];
```

```

        for (int j = 0; j < cols; j++)
            arr[i][j] = other.arr[i][j];
    }
}

Matrix operator+(const Matrix &other)
{
    if (rows != other.rows || cols != other.cols)
    {
        cout << "Matrices must have the same dimensions for addition." << endl;
        return Matrix(0, 0);
    }

    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            result.arr[i][j] = arr[i][j] + other.arr[i][j];
    return result;
}

Matrix operator-(const Matrix &other)
{
    if (rows != other.rows || cols != other.cols)
    {
        cout << "Matrices must have the same dimensions for subtraction." <<
endl;
        return Matrix(0, 0);
    }

    Matrix result(rows, cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            result.arr[i][j] = arr[i][j] - other.arr[i][j];
    return result;
}

Matrix operator*(const Matrix &other)
{
    if (cols != other.rows)
    {
        cout << "Number of columns of first matrix must equal number of rows of
second matrix for multiplication." << endl;
        return Matrix(0, 0);
    }

    Matrix result(rows, other.cols);
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < other.cols; j++)

```

```

        for (int k = 0; k < cols; k++)
            result.arr[i][j] += arr[i][k] * other.arr[k][j];
    return result;
}

int *operator[](int index)
{
    return arr[index];
}

~Matrix()
{
    for (int i = 0; i < rows; i++)
    {
        delete[] arr[i];
    }
    delete[] arr;
}

void input()
{
    cout << "Enter elements for a " << rows << "x" << cols << " matrix:" << endl;
    for (int i = 0; i < rows; i++)
        for (int j = 0; j < cols; j++)
            cin >> arr[i][j];
}

void print()
{
    for (int i = 0; i < rows; i++)
    {
        for (int j = 0; j < cols; j++)
        {
            cout << arr[i][j] << " ";
        }
        cout << endl;
    }
}

friend bool search(const Matrix&, int);
};

bool search(const Matrix &mat, int target)
{
    int left = 0, right = mat.rows * mat.cols - 1;
    while (left <= right) {
        int middle = (left+right) / 2;
        int r = middle / mat.cols;
        int c = middle % mat.cols;

        if (target == mat.arr[r][c])

```



```

        return true;
    else if (target < mat.arr[r][c])
        right = middle - 1;
    else
        left = middle + 1;
    }
    return false;
}

int main() {
    int m, n, target;
    cout << "Enter dimension (m x n):" << endl;
    cout << "m: ";
    cin >> m;
    cout << "n: ";
    cin >> n;
    Matrix A(m, n);
    A.input();
    cout << "Enter the element u want to search: ";
    cin >> target;
    cout << "target in matrix? " << (search(A, target) ? "true :)" : "false :(") <<
endl;
    return 0;
}

```

OUTPUT:

```

Enter dimension (m x n):
m: 3
n: 4
Enter elements for a 3x4 matrix:
1 3 5 7
10 11 16 20
23 30 34 60
Enter the element u want to search: 3
target in matrix? true :)

```

QUESTION#3:

CODE:

```
#include <iostream>
using namespace std;

class Employee {
private:
    string name, desg;
    double salary;
    const int id;
public:
    Employee() : name("N/A"), desg("N/A"), id(0), salary(0) {}
    Employee(string name, string desg, int id, double salary) : id(id) {
        this->name = name;
        this->desg = desg;
        this->salary = salary;
    }
    void details() {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Designation: " << desg << endl;
        cout << "Salary: " << salary << endl<<endl;
    }

    friend class Payroll;
};

class Payroll {
public:
    void setSalary(Employee &emp, double salary) {
        emp.salary = salary;
    }
    void incSalary(Employee &emp, double percent) {
        emp.salary += emp.salary*percent/100;
    }
};

int main() {
    Employee emp("ABC", "Data Scientist", 5248, 500000);
    emp.details();

    Payroll p;
    p.setSalary(emp, 600000);

    cout << "Setting Salary:"<< endl;
```

```
    emp.details();  
  
    p.incSalary(emp, 10);  
    cout << "Increasing salary by 10%:"<<endl;  
    emp.details();  
    return 0;  
}
```

OUTPUT:

```
Name: ABC  
ID: 5248  
Designation: Data Scientist  
Salary: 500000  
  
Setting Salary:  
Name: ABC  
ID: 5248  
Designation: Data Scientist  
Salary: 600000  
  
Increasing salary by 10%:  
Name: ABC  
ID: 5248  
Designation: Data Scientist  
Salary: 660000
```

QUESTION#4:

CODE:

```
#include <iostream>
using namespace std;

class Employee
{
private:
    string name, desg;
    double salary;
    const int id;

public:
    Employee() : name("N/A"), desg("N/A"), id(0), salary(0) {}
    Employee(string name, string desg, int id, double salary) : id(id)
    {
        this->name = name;
        this->desg = desg;
        this->salary = salary;
    }
    void details()
    {
        cout << "Name: " << name << endl;
        cout << "ID: " << id << endl;
        cout << "Designation: " << desg << endl;
        cout << "Salary: " << salary << endl
            << endl;
    }

    friend void setSalary(Employee &, double);
    friend void incSalary(Employee &, double);
};

void setSalary(Employee &emp, double salary)
{
    emp.salary = salary;
}

void incSalary(Employee &emp, double percent)
{
    emp.salary += emp.salary * percent / 100;
}

int main()
{
    Employee emp("ABC", "Data Scientist", 5248, 500000);
```

```
    emp.details();

    setSalary(emp, 600000);

    cout << "Setting Salary:" << endl;
    emp.details();

    incSalary(emp, 10);
    cout << "Increasing salary by 10%:" << endl;
    emp.details();
    return 0;
}
```

OUTPUT:

```
Name: ABC
ID: 5248
Designation: Data Scientist
Salary: 500000

Setting Salary:
Name: ABC
ID: 5248
Designation: Data Scientist
Salary: 600000

Increasing salary by 10%:
Name: ABC
ID: 5248
Designation: Data Scientist
Salary: 660000
```